

EVC - Set size increase

Smart Contract Security Assessment

Contents

1	Review Summary	2
1.1	Protocol Overview	2
1.2	Audit Scope	2
1.3	Risk Assessment Framework	2
1.3.1	Severity Classification	2
1.4	Key Findings	3
1.5	Overall Assessment	3
2	Audit Overview	3
2.1	Project Information	3
2.2	Audit Team	3
2.3	Audit Timeline	3
2.4	Audit Resources	4
2.5	Critical Findings	4
2.6	High Findings	4
2.7	Medium Findings	4
2.8	Low Findings	4
2.9	Gas Savings Findings	4
2.10	Informational Findings	4
2.10.1	Raising <code>SET_MAX_ELEMENTS</code> amplifies computation, indexing and bad debt risks	4
2.10.2	Analysis of Set size during EVK liquidations	5
2.10.3	Tests still encode the previous 10 element Set limit	6
2.10.4	Interface documentation still describes a 10 element Set limit	7

1 Review Summary

1.1 Protocol Overview

The EVC (Ethereum Vault Connector) is the foundation of the EVK (Ethereum Vault Kit) from Euler. It connects vaults together with a robust and flexible framework for developers to build upon. Gondor is leveraging the EVC and EVK to integrate with Polymarket tokens; the EVC change is required to allow a higher number of collaterals.

1.2 Audit Scope

This audit covers a single-line change in `src/Set.sol`, reviewed in the context of the surrounding EVC codebase, across a half day of review.

`src/Set.sol`

`src/EthereumVaultConnector.sol`

1.3 Risk Assessment Framework

1.3.1 Severity Classification

Severity	Description	Potential Impact
Critical	Immediate threat to user funds or protocol integrity	Direct loss of funds, protocol compromise
High	Significant security risk requiring urgent attention	Potential fund loss, major functionality disruption
Medium	Important issue that should be addressed	Limited fund risk, functionality concerns
Low	Minor issue with minimal impact	Best practice violations, minor inefficiencies
Undetermined	Findings whose impact could not be fully assessed within the time constraints of the engagement. These issues may range from low to critical severity, and although their exact consequences remain uncertain, they present a sufficient potential risk to warrant attention and remediation.	Varies based on actual severity
Gas	Findings that can improve the gas efficiency of the contracts.	Increased transaction costs
Informational	Code quality and best practice recommendations	Reduced maintainability and readability

Table 1: severity classification

1.4 Key Findings

Breakdown of Finding Impacts

Impact Level	Count
■ Critical	0
■ High	0
■ Medium	0
■ Low	0
■ Informational	4

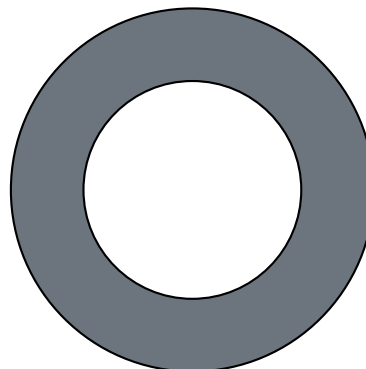


Figure 1: Distribution of security findings by impact level

1.5 Overall Assessment

The change to `SET_MAX_ELEMENTS` is sound and did not yield any serious findings. However, auditors note that combined with the Polymarket integration and the larger collateral fan-out, it will lead to significantly higher gas consumption, additional indexing complexity, and amplified lending risk on the controller side. Additionally, most tests and the Certora formal verification rules were already configured with a lower loop bound than the previous cap of 10, and have not been re-tuned for the new cap of 100.

2 Audit Overview

2.1 Project Information

Protocol Name: Gondor

Repository: <https://github.com/euler-xyz/ethereum-vault-connector/>

Commit Hash: dd04b59071525425cab2960ede5d7495847d87ee

Commit URL: <https://github.com/euler-xyz/ethereum-vault-connector/commit/dd04b59071525425cab2960ede5d7495847d87ee>

2.2 Audit Team

Adriro, HHK

2.3 Audit Timeline

The audit was conducted from May 11 to 11, 2026.

2.4 Audit Resources

- Code repositories and documentation

2.5 Critical Findings

None.

2.6 High Findings

None.

2.7 Medium Findings

None.

2.8 Low Findings

None.

2.9 Gas Savings Findings

None.

2.10 Informational Findings

2.10.1 Raising `SET_MAX_ELEMENTS` amplifies computation, indexing and bad debt risks

Raising `SET_MAX_ELEMENTS` from 10 to 100 ([Set.sol#L7](#)) scales every path that touches an EVC set linearly with the new cap: liquidation gas, account-status checks, `controlCollateral` membership checks, collateral management, and off-chain indexing of per-account collateral state.

It also increases the potential number of liquidations needed to bring a sub-account back to a positive health factor.

Technical Details

The controller's `checkAccountStatus` ([EthereumVaultConnector.sol#L935-L960](#)) iterates the full collateral array, and `controlCollateral` performs a linear `contains` check on each seizure ([EthereumVaultConnector.sol#L584](#)). `Set.insert` / `remove` / `contains` are O(n) linear scans, so `enableCollateral` / `disableCollateral` / `enableController` / `isCollateralEnabled` / `isControllerEnabled` all scale with the set size as well. Additionally, each enabled collateral may need to be liquidated to keep the sub-account healthy, potentially requiring up to 100 liquidations per sub-account, which could be hard to execute during volatile times.

Impact

Informational. The increase does not break invariants or deployability on the target chain (Polygon), but the per-operation cost, indexing overhead, and bad debt risk increase significantly.

Recommendation

Document the trade-off and consider lowering the limit or whitelisting only a handful of collateral vaults.

Developer Response

Acknowledged.

2.10.2 Analysis of Set size during EVK liquidations

Increasing the EVC Set maximum from 10 to 100 expands the number of enabled collaterals that an account can carry. Liquidation remains bounded, but a borrower can increase liquidation gas by filling the collateral set, especially with recognized collateral vaults that hold dust balances.

Technical Details

EVK relies on the EVC collateral set during risk checks and liquidations. During `liquidate()`, the liquidating vault calls `calculateLiquidation()`, which fetches the violator's full collateral list with `getCollaterals()`. In EVC, this copies the full collateral Set. `calculateLiquidation()` then verifies that the selected collateral is enabled with `isCollateralEnabled()`, which performs a linear EVC `contains()` lookup over the same Set.

The max liquidation calculation then calls `calculateLiquidity()` over the full collateral array. Each collateral is inspected with `getCollateralValue()`, which loads the LTV configuration, checks the collateral balance, and prices nonzero recognized balances through the configured oracle. If debt socialization is considered after the collateral transfer, `checkNoCollateral()` performs another full loop over the same collateral list to determine whether any recognized collateral balance remains. The collateral seizure itself uses `evc.controlCollateral()` through `enforceCollateralTransfer()`, and EVC again checks that the seized collateral is enabled for the violator before making the controlled transfer. This means the EVC Set size affects liquidations in several places: EVC copies the collateral Set, EVC performs linear membership checks, EVK loops the returned collateral array for liquidity, EVK may loop it again for debt socialization, and EVC checks collateral membership again during `controlCollateral()`. Unrecognized collateral addresses are comparatively cheap because `getCollateralValue()` returns when LTV is zero. Recognized collaterals with nonzero dust balances are more expensive because each entry runs balance and oracle logic. The larger bound gives borrowers more room to increase the cost of liquidations. This can matter for small positions or marginal liquidation rewards, where the additional gas can reduce keeper profitability. A benchmark that measures liquidations under different configurations is available [here](#).

Scenario	Liquidation gas
1 collateral	248,813
10 unrecognized collaterals	238,187
50 unrecognized collaterals	375,610
25 recognized dust collaterals	524,547
50 recognized dust collaterals	861,456
75 recognized dust collaterals	1,198,573
100 recognized dust collaterals	1,535,896

The recognized dust case is the more relevant stress case for liquidation incentives. It models an account with the full collateral set where each extra collateral is recognized by the controller vault and has a nonzero balance, forcing EVK to evaluate each collateral rather than skipping it as unrecognized or empty.

A controller vault configured to accept 100 different collateral vaults with nonzero LTVs is expected to be uncommon in normal use. The benchmark should therefore be interpreted as an adversarial upper-bound style test for the increased Set size rather than a typical liquidation profile. It also does not fully model production oracle costs. The benchmark uses mocked oracle pricing, while real oracle routing and quoting can add materially more gas per recognized collateral.

Impact

Informational.

Recommendation

Consider adding gas benchmarks for representative production oracle and collateral configurations, including recognized dust collateral sets and debt socialization paths, that model worst-case adversarial scenarios.

Developer Response

Acknowledged.

2.10.3 Tests still encode the previous 10 element Set limit

Several EVC test and specification files still encode the old 10-element Set limit even though `SET_MAX_ELEMENTS` is now 100. This can leave the new boundary under-tested and can cause formal or gas-oriented checks to validate behavior against the previous cap.

Technical Details

`test/evc/EthereumVaultConnectorScribble.sol` contains Scribble annotations that still bind `accountStatusChecks` and `vaultStatusChecks` to 10 elements. The same file also

uses `old(accountCollaterals[account].numElements) < 10` and `old(accountControllers[account].numElements) < 10` in postconditions for `enableCollateral()` and `enableController()`. These annotations no longer match the Set implementation and do not cover the enlarged 11 to 100 element range. The executable EVC unit and invariant tests mostly use `SET_MAX_ELEMENTS`, so the new limit is eligible for fuzzing in paths such as collateral management, deferred account checks, deferred vault checks, and combined account and vault checks. However, there are no deterministic EVC boundary tests that force exactly 100 collaterals, 100 deferred account checks, or 100 deferred vault checks. `test/gas/SetGas.t.sol` still constructs a `size10` set and expects insertion of the 11th element to revert. With `SET_MAX_ELEMENTS` set to 100, this expectation is stale and does not measure the new worst-case costs for full-set duplicate checks, misses, removals, or overflow insertion.

Impact

Informational.

Recommendation

Update the EVC Scribble annotations to use `SET_MAX_ELEMENTS` or the literal value 100 where Scribble cannot resolve the constant. Add deterministic tests for exactly full 100-element sets in the EVC paths that rely on collaterals and deferred status checks. Update the Set gas tests to include a full-size 100-element case and expect `TooManyElements` only on the 101st insertion.

Developer Response

Addressed here: [PR#203](#).

2.10.4 Interface documentation still describes a 10 element Set limit

`IEthereumVaultConnector` still documents several Set-backed limits as 10 elements even though `SET_MAX_ELEMENTS` has been increased to 100. This can mislead integrators and reviewers about the maximum number of collaterals, transient controllers, deferred account checks, and deferred vault checks supported by the EVC.

Technical Details

`src/Set.sol` defines `SET_MAX_ELEMENTS` as 100, and the EVC uses the same Set implementation for `accountCollaterals`, `accountControllers`, `accountStatusChecks`, and `vaultStatusChecks`.

The interface comments for `enableCollateral()`, `enableController()`, `requireAccountStatusCheck()`, and `requireVaultStatusCheck()` still state that there can be at most 10 unique elements in the corresponding set. These comments no longer match the implementation and can cause off-chain tooling, integrations, test assumptions, and protocol documentation to be built around the previous lower bound.

Impact

Informational.

Recommendation

Update the stale mentions in `IEthereumVaultConnector.sol` from 10 to 100, or refer to `SET_MAX_ELEMENTS` conceptually if the comments should remain resilient to future constant changes.

Developer Response

Addressed here: [PR#203](#).



EVC - Set size increase

Completed 2026-05-11